

GL1F: Reproducible Integer Tree-Ensemble Inference on the EVM

Mikhail Fedorov^{*†}

Abstract. Public model storage does not by itself establish which function a registered model executes. We study this gap for fixed-depth integer tree ensembles on the Ethereum Virtual Machine (EVM). GL1F combines a canonical scalar/vector encoding, ordered code-as-data chunks, and separate local and contract evaluators. We specify sufficient conditions for exact reconstruction and integer execution agreement, including registry/header consistency and boundary-spanning reads. An executable counterexample shows that registration alone does not bind an identifier to the referenced bytes. JavaScript, Python, and C++ trainers emit identical cores for 25 tested profiles; arithmetic counterexamples delimit that empirical result. Across six scalar shapes, code-as-data publication uses 61.8–69.4% less gas than packed Solidity storage, with mixed inference costs. A pinned-state archive makes 108 output comparisons for 12 models totaling 31,185,324 bytes replayable without a live provider. The resulting contribution is a reproducible method for checking the connection between a public model identifier, its bytes, and its executed integer function. It establishes conditional execution agreement, without establishing predictive validity or authenticating provider-reported chain history.

1. Introduction

Publishing a trained model does not fully specify a reproducible prediction. A result also depends on feature ordering, preprocessing, numeric conventions, serialization, and the implementation that interprets the artifact. Reproducibility initiatives in computational science and machine learning therefore emphasize executable artifacts and explicit evaluation protocols.^{1,2} A blockchain supplies a persistent, ordered record of public data and state transitions, but the association between a model identifier and an executed function still requires verification. A registry can contain inconsistent metadata; a decoder can interpret malformed bytes differently; and a remote procedure call (RPC) can report a result without providing evidence that authenticates the underlying state.

This paper addresses a bounded version of that problem: under which representation, storage, state, input, and arithmetic conditions do local and EVM evaluators produce exactly the same integer result for a registered tree ensemble? The question is relevant when a contract must consume the model output directly, or when researchers need to reconstruct the precise public function available at an earlier block. For applications requiring only local arithmetic replay, a

^{*} Corresponding author.

[†] M. Fedorov is the founder of GenesisL1.

content-addressed file and a specified evaluator can suffice. The additional role of the ledger is to order public artifacts and service state and make them accessible to contract execution.

Gradient-boosted decision trees provide a useful restricted model family. They are widely used for tabular learning,^{3–5} but their inference kernel needs only comparisons, indexed leaf selection, and addition. Complete fixed-depth trees make both offsets and work counts explicit. Integer thresholds, leaves, and packed inputs avoid floating-point ambiguity during inference. These properties support a small execution specification, while retaining scalar scores and vector logits for several training objectives.

GLIF implements that design with browser, Python, and C++ trainers, a canonical binary model format, and a Solidity interpreter. Models exceeding a single contract’s code capacity are split across public code-as-data objects. An ordered address table and registry record identify the storage layout. The implementation is deployed on GenesisL1, an EVM-compatible network. The model bytes are intentionally public, allowing reconstruction and local inference even when a canonical service endpoint is metered.

The contribution is the composition of three checkable relations. First, an ordered storage manifest must reconstruct one exact canonical byte string. Second, the registry fields actually consumed by the runtime must agree with that string. Third, evaluators must apply the same signed decoding, strict-greater branch rule, and exact accumulation to the same packed input. We state sufficient conditions for these relations and provide a short specification-level equivalence proof. An executable counterexample establishes why the registered identifier additionally needs an explicit content-hash check. Cross-language and local-EVM tests then assess the implemented paths, including numeric and chunk-boundary failures. A controlled storage experiment and a pinned deployment archive evaluate the cost and practical observability of this construction.

Exact on-chain tree inference, quantized parameters, and code-as-data storage are established techniques.^{6,7} The present study examines the connection from a registry-named collection of objects to its executed integer function, rather than claiming priority for those components. Its results also distinguish two forms of reproducibility: inference equality follows under stated premises, whereas training equality is supported only for the executed profiles and numeric environment. The arithmetic counterexamples are informative precisely because they prevent a finite test matrix from being mistaken for a universal training guarantee.

The evaluation concerns execution assurance. It does not test whether the published models predict their application targets accurately. Throughout, “independent evaluator” means a separately implemented checking path; the author developed the system and conducted the experiments, so this is not an externally replicated study. This distinction permits specific, falsifiable claims about representation and execution while leaving model quality and chain-state authentication to their own evidence.

2. Related work

2.1. Portable and blockchain-executed models—Gradient boosting constructs additive predictors by fitting successive weak learners to a loss-derived target.³ XGBoost and LightGBM provide mature training systems with regularization, histogram methods, and scalable execution.^{4,5} GLIF restricts tree shape for explicit offsets and bounded inference work; the experiments here do not compare its predictive performance with those systems.

ONNX already defines a portable tree-ensemble operator, including branch modes, leaf targets, aggregation, and post-transforms.⁸ Hummingbird compiles conventional machine-learning pipelines to tensor computations for prediction serving.⁹ GLIF’s smaller grammar provides a fixed-width integer object that can be interpreted directly by the EVM. The tradeoff is reduced model and operator flexibility, not a general replacement for model interchange.

Prior blockchain systems have evaluated collaborative model updating on Ethereum, logistic-regression inference in Fabric, integer Naive Bayes in Solidity, and off-chain-trained bagged trees deployed as prediction contracts.^{10–13} ML2SC translates PyTorch multilayer perceptrons to Solidity, transfers their parameters, applies fixed-point arithmetic, and studies gas costs.¹⁴ These results establish that off-chain training followed by contract inference is not specific to GLIF.

The closest property-level overlap is Alhaidari et al., who report quantized, serialized, hash-bound model parameters, read-call and transaction inference, gas analysis, and reported Z3 verification of bit-exact model execution, with decision-tree support.⁶ Consequently, bit-exact tree execution and parameter hash binding cannot serve as standalone novelty claims here. GLIF instead supplies a strict reconstruction predicate for an ordered multi-object artifact, reconciles it with the metadata consumed by a runtime, and demonstrates a concrete case in which a registry key fails to bind the referenced model. Its scalar/vector grammar, conformance matrix, and replayable deployment record make that composition inspectable.

2.2. Storage, verification, and provenance—The EVM exposes deterministic execution and resource charging through gas.¹⁵ EIP-170 limits deployed runtime code to 24,576 bytes.¹⁶ Code-as-data libraries such as SSTORE2 already use contract creation to publish bytes and `EXTCODECOPY` to recover them.⁷ The storage primitive is therefore prior art. Here the relevant object is the ordered table of chunk addresses, exact framing and lengths, and the boundary reader used by a model interpreter. The controlled experiment isolates a practical tradeoff between that design and a packed Solidity-storage implementation.

Fu et al. authenticate binary decision-tree predictions using committed trees and provider-supplied path proofs.¹⁷ Zero-knowledge systems prove decision-tree predictions or accuracy while protecting selected private information,^{18,19} whereas opML uses off-chain execution and an optimistic dispute protocol.²⁰ GLIF directly interprets public weights. It requires no per-prediction prover or challenge window, but it incurs replicated execution and provides no model confidentiality. The experiments do not rank those architectures on a common workload.

KEVM gives a formal semantics of EVM bytecode.²¹ The theorem in this paper relates conforming evaluator specifications; it is not a KEVM proof, a verified compiler translation, or a mechanized proof of the deployed bytecode. Source inspection and finite execution tests support the separate implementation-conformance claim.

Supply-chain provenance systems address additional questions. *in-toto* links authorized processing steps and their materials,²² while MLflow2PROV records provenance across machine-learning pipelines.²³ A content digest in GLIF identifies bytes but does not identify their trainer, establish dataset lineage, or authenticate a scientific interpretation. Likewise, a fixed compiler configuration does not by itself demonstrate reproducible builds.²⁴

The public GLIF protocol document dated 19 January 2026 already describes the alpha format, storage architecture, contracts, and browser workflow.²⁵ The present article evaluates that system with explicit verification conditions, a registry counterexample, adversarial cross-language conformance tests, and quantitative storage and deployment evidence. The earlier

document remains identifiable as prior system documentation.

3. Representation and system design

3.1. Canonical integer forest—Let $\mathbb{I}_{32} = [-2^{31}, 2^{31} - 1] \cap \mathbb{Z}$. A complete binary tree of depth d has $I(d) = 2^d - 1$ internal nodes and $L(d) = 2^d$ leaves. Each internal node occupies eight bytes: an unsigned 16-bit feature index, a signed 32-bit threshold, and two zero reserved bytes. Each leaf is a signed 32-bit increment. Multibyte fields are little-endian, and nodes use flat heap order. The tree length is therefore

$$b(d) = 8(2^d - 1) + 4 \cdot 2^d = 12 \cdot 2^d - 8. \quad (1)$$

A 24-byte fixed header records the version, feature count, depth, tree count, base or output count, and positive quantization scale Q .

Version 1 represents a scalar ensemble with base $a_0 \in \mathbb{I}_{32}$ and T trees. Version 2 represents $K \geq 2$ outputs, $R \geq 1$ trees per output, and a base vector in $\mathbb{I}_{32}^K$. Its trees are output-major, and its base vector follows the fixed header. The canonical lengths are

$$|B_{v1}| = 24 + Tb(d), \quad |B_{v2}| = 24 + 4K + KRb(d). \quad (2)$$

These are representation identities rather than statistical model-size estimates. A canonical parser checks the magic, recognized version, zero reserved fields, positive feature count and scale, exact length, supported depth, and every feature index. The evaluated deployment profile uses $1 \leq d \leq 12$. For a record satisfying registry/core agreement, the registry’s 16-bit total-tree field requires $T \leq 65535$ for v1 and $KR \leq 65535$ for v2. Those bounds are narrower than the 32-bit tree-count fields in the core. Version 1 admits a base-only object with $T = 0$; trained evaluation fixtures use positive tree counts. Exact header offsets appear in the supplement and the repository format specification.

An input is exactly $4F$ bytes containing $q \in \mathbb{I}_{32}^F$. It has already been quantized and ordered before it reaches the inference interface. Starting at heap index $i_0 = 0$, each decision is

$$i_{j+1} = 2i_j + 1 + \mathbf{1}[q_{f(i_j)} > \tau(i_j)]. \quad (3)$$

Equality goes left. After d decisions, the selected leaf is indexed by $i_d - I(d)$. The scalar function and each vector component are

$$f_B(q) = a_0 + \sum_{t=0}^{T-1} v_{t, \pi_t(q)}, \quad f_{B,k}(q) = a_{0,k} + \sum_{t=0}^{R-1} v_{k,t, \pi_{k,t}(q)}. \quad (4)$$

A class endpoint returns the lowest index among equal maximal integer logits. Probabilities, calibration, and feature transformations belong to application code, outside this packed-input function.

The core is an artifact identity, not a semantic normal form: distinct cores can implement the same function. Conversely, identical scalar bytes may be labelled as a regression score or a binary logit. The core does not encode feature names, units, dataset provenance, or a training-task discriminator. Optional GLIF JSON metadata is excluded from core hashing. A scientific interpretation therefore needs a separately authenticated record of preprocessing, feature and label order, and intended output meaning.

3.2. Publication and reconstruction—`ModelStore` publishes runtime code of the form `GL1C||payload`. Its four-byte prefix leaves at most 24,572 payload bytes within EIP-170. A second code object contains an ordered table of 32-byte address slots; the low 20 bytes identify a chunk and the high 12 bytes must be zero. One such table can hold at most $\lfloor 24572/32 \rfloor = 767$ addresses. For a core of length ℓ , a nominal payload stride c , and $N = \lceil \ell/c \rceil$ chunks, the final chunk may be shorter than c ; every earlier chunk must have exactly c payload bytes.

The registry records the table pointer, byte length, stride, chunk count, and inference fields. `ForestRuntime` reads signed and unsigned primitives through `EXTCODECOPY`. A read crossing a chunk boundary copies its prefix from one payload and its suffix from the next. Requiring $c \geq 4$ ensures that each one-, two-, or four-byte primitive spans at most two chunks. The prefix `GL1C` is a framing marker, not an authenticator: correctness also requires the exact table, order, lengths, and payloads.

The publication workflow serializes a core, validates it locally, computes Ethereum Keccak-256, stores its chunks and address table, and registers the declared metadata. Verification reads those objects back at a selected state, reconstructs the exact core, repeats canonical parsing, reconciles registry fields, and checks the digest against the requested identifier. SHA-256 is also recorded for ordinary file comparisons. Ethereum Keccak-256 here means the legacy Ethereum hash, not FIPS SHA3-256.

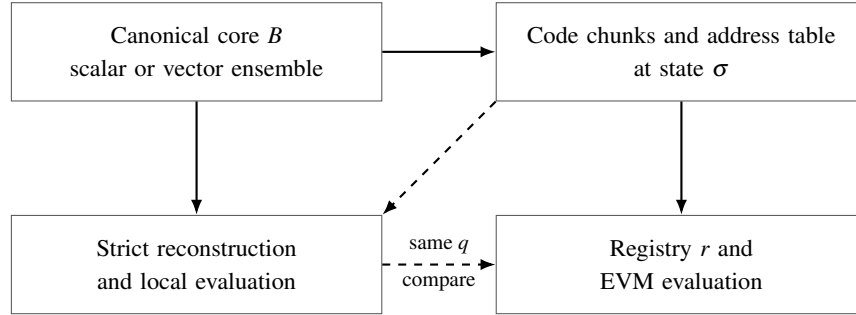


Fig. 1. Publication and verification paths. Solid arrows indicate artifact or execution flow; dashed arrows indicate reconstruction and comparison. Verification checks canonical bytes, ordered storage, registry agreement, and content binding before comparing the integer outputs for the same packed input q .

3.3. Mutable service state—The model NFT and optional marketplace surround the public artifact with ownership, discovery, authorization, and payment services. These services can change while the model bytes remain unchanged. In particular, token ownership, endpoint access, and the fee destination are distinct registry relations. An NFT transfer does not establish authorship or validate the model’s dataset. The evaluated NFT implements a custom ownership interface; full ERC-721 metadata and discovery interoperability is not assumed.²⁶

A paid endpoint can meter its own execution or service integration. It cannot prevent anyone who reconstructs the public bytes from evaluating them locally. A transaction exposes its packed input as public calldata. A view call avoids that transaction but still reveals the request to the RPC provider. Authorization therefore controls an endpoint rather than providing input or model confidentiality.

4. Execution assurance

4.1. Sufficient conditions—Consider one EVM state σ , registry record r , core B , and manifest $M = (P, c, N, \ell)$. Call the manifest well formed when $P \neq 0$, $4 \leq c \leq 24572$, $1 \leq N \leq 767$, $\ell = |B|$, and $N = \lceil \ell/c \rceil$; the table code is exactly $4 + 32N$ bytes with the required prefix and canonical nonzero address slots; and chunk j is exactly $\text{GL1C} \| B[jc : \min((j+1)c, \ell)]$. All objects and registry getters must refer to the same state. Exact final lengths exclude both truncation and undeclared trailing payload.

The verifier reconciles nine registry/core/storage relations. The inference fields are feature count F , total tree count (T or KR), depth d , scalar base (a_0 for v1, zero for the v2 reserved-field convention), and scale Q . The storage fields are table pointer P , total bytes ℓ , number of chunks N , and stride c . For v2 the actual base vector comes from the bytes after the header. This is a sufficient verification predicate; not every listed field is consumed by every endpoint’s arithmetic.

A conforming evaluator decodes the specified little-endian signed and unsigned values, follows Equation 3, and computes the finite integer sums without overflow or precision loss. A conforming local evaluator first validates the canonical core. A conforming EVM evaluator obtains its primitives from the manifest and its runtime metadata from the reconciled record. Endpoint authorization, enabled-state, and payment conditions must be satisfied so that execution reaches this evaluator with the unchanged packed input.

Theorem 4.1 (Conditional local/EVM execution agreement). *Let B be canonical, let M be well formed for B at (σ, r) , and let the nine registry/core/storage relations hold. Given the same $4F$ packed bytes for $q \in \mathbb{I}_{32}^F$, conforming local and EVM evaluators return the same scalar or vector in Equation 4. Their class endpoints return the same lowest-index maximizer and maximal logit.*

Proof. Write a primitive offset as $o = jc + s$, with $0 \leq s < c$. If its length n ends within the chunk, the reader returns exactly $B[o : o + n]$. Otherwise it joins $c - s$ bytes from chunk j and the remaining $n - (c - s)$ bytes from chunk $j + 1$. Since $n \leq 4 \leq c$, a third chunk is unnecessary. Exact manifest framing and lengths therefore supply identical primitive bytes to both evaluators. Registry agreement supplies the same metadata where the contract obtains it from the record rather than the core header.

At each root both evaluators start at index zero. If their indices agree before a decision, they read the same feature and threshold and apply the same strict comparison, so their next indices agree. Induction over d levels selects the same leaf. Indeed, after j decisions, $2^j - 1 \leq i_j \leq 2^{j+1} - 2$, so subtracting $I(d)$ at level d yields a valid leaf index. Repeating over the finite ensemble gives identical summands. Exact accumulation yields identical outputs, and the shared tie rule yields identical class results. ■

The theorem establishes an implication between specified objects and algorithms. It does not prove that arbitrary Solidity bytecode conforms, that an RPC returned the authentic state, or that a named record contains its claimed bytes. The tests below address enumerated implementation paths. The supplement gives the full geometry, traversal, reconstruction, and quantization proofs without treating their elementary components as separate algorithmic inventions.

4.2. Content binding and a counterexample—Execution agreement for reconstructed bytes does not require their registered name to be their digest. If the name is claimed as a content

identifier, the verifier must additionally establish

$$\text{modelId} = \text{keccak256}(B). \quad (5)$$

The deployed registry accepts the identifier and storage tuple as separate arguments and does not reconstruct and hash the core during registration. An admissible nonzero unused identifier different from the core digest is therefore a counterexample to unconditional binding.

The isolated-EVM test instantiates this case against the unmodified contracts. A valid 104-byte v1 object was registered under an identifier different from its computed Keccak-256 digest. The runtime followed that record and returned 1043, the referenced object’s prediction. The production JavaScript loader reconstructed the same bytes, detected the mismatch, and rejected the record. The complete identifiers and executable fixture are included in the supplement and integration record. This is a client-enforced verification condition; it does not introduce an invariant into already deployed bytecode.

This distinction generalizes beyond a particular tree format. A registry lookup, a content digest, and execution under metadata are different links in a computation’s provenance. A positive observation for all records at one block cannot eliminate a counterexample allowed by the registration interface. Verification must check each link for the record being used.

4.3. Bounds and approximation—Scalar inference performs exactly Td decisions and T leaf reads; vector inference performs KRd decisions and KR leaf reads. An abstract streaming evaluator needs constant scalar traversal state or $O(K)$ output state, excluding its inputs and read buffers. Finite work does not imply execution within a particular gas limit. Moreover, the current Solidity boundary reader allocates fresh buffers for crossing primitives; EVM memory can grow during a call even though each primitive is small.

Under registry/core agreement, the count bound makes scalar sums and every partial sum no larger in magnitude than $(65535 + 1)2^{31} = 2^{47}$. Since $KR \leq 65535$, each vector component is bounded at least as tightly. These are below JavaScript’s 2^{53} exact-integer boundary and far below int256 limits. They apply to the documented deployment profile, not arbitrarily large external files.

For a finite real value z and positive integer Q , the abstract quantizer is

$$\mathcal{Q}_Q(z) = \text{clamp}_{\mathbb{I}_{32}} \left(\lfloor Qz + \frac{1}{2} \rfloor \right). \quad (6)$$

It rounds half ties toward positive infinity. Without saturation, the single-value error is at most $1/(2Q)$. If quantizing a base and the selected leaves leaves every path unchanged, scalar output error is at most $(T + 1)/(2Q)$; replace T by R for one vector component. A split margin $|x - \tau| > 1/Q$ suffices to preserve that split under this abstract quantizer. These elementary bounds concern quantization of a fixed forest, not the error between separately trained models.

Production preprocessing first computes a binary64 product $\text{fl}(Qz) = Qz + \delta_Q(z)$ and then rounds. Its unsaturated single-value bound is $(1/2 + |\delta_Q(z)|)/Q$. The abstract half-unit bound therefore requires exact agreement with the abstract quantizer or this product-rounding correction. Saturation and a changed path invalidate the stated stable-path bound; a small feature perturbation near a threshold can select a substantially different leaf.

5. Implementation and experimental methods

5.1. Training and conformance profile—The three trainers support regression, binary, multiclass, and multilabel objectives with histogram-based fixed-depth trees. They use seeded `xorshift32`, linear or quantile bins, explicit feature and output ordering, and shared serialization. When a split is unavailable, a complete continuation duplicates the terminal value and uses feature zero with the maximal `int32` threshold. It preserves the terminal result and fixed-depth geometry for every packed input.

Training uses floating-point gradients, split scores, metrics, exponentials, and logarithms. The publication profile fixes a finite `float32` input matrix, row and feature order, target representation, hyperparameters, seed, random-consumption order, and numeric operation sequence. The C++ build uses `-ffp-contract=off -fno-fast-math`. These restrictions allow meaningful byte-level comparisons without asserting that every platform or mathematical library will emit the same model. Comparisons exclude optional JSON metadata containing timestamps.

Three observed discrepancies motivate the profile. First, a standalone linear-binning witness selects bin four using division by the range and bin three using multiplication by a rounded cached reciprocal, although the expressions are algebraically equivalent. Second, a generated 4,000-row, 12-feature, four-class fixture exposed a one-byte core difference when the softmax denominator accumulated binary32-rounded rather than binary64 exponentials. Third, evaluating $\lfloor z + 1/2 \rfloor$ with a binary64 addition rounded a value immediately below a half boundary upward before the floor. The Python and C++ rounding implementations now compare the fractional part with $1/2$, matching JavaScript’s finite rounding convention. The operation order, softmax precision, half-boundary, and explicit seed-zero cases are retained as regression controls. These corrections close the observed failures and remain empirically testable.

5.2. Evaluation design—The evaluation separates four questions: whether implemented parsers and evaluators agree at format and arithmetic boundaries; whether the three trainers emit identical cores for declared profiles; how identical ensembles compare under two EVM storage implementations; and whether a public deployment can be reconstructed and its reported outputs replayed from an archived state observation. The synthetic fixtures target execution behavior and contain no claim about a predictive application.

The parity dataset has 240 rows and five mixed-scale finite features, with deterministically generated regression, binary, three-class, and three-label targets. Baseline profiles use 18 depth-3 trees, learning rate 0.075, minimum leaf size four, 16 bins, $Q = 100000$, a 70/20/10 split, and seed 20260724. The tests compare entire canonical core byte strings, rather than only approximate outputs. The checking parser in `tests/publication/model_format.py` and the bigint inference oracle are separate implementations from the production decoder.

The local EVM experiments compile the production contracts with Solidity 0.8.20, optimizer 200, `viaIR=true`, and `evmVersion=istanbul`.²⁷ They execute in Ganache 7.9.2 under its recorded hardfork configuration.²⁸ The production runtime compiles to 17,794 runtime bytes, below EIP-170. The compiler profile is deliberately pinned because a permissive source pragma is not evidence that all compiler versions build the same program.

The recorded CPU study uses Python 3.12.13, NumPy 2.3.5,²⁹ Node 24.19.0, and g++ 13.3.0 on Linux x86-64 with an AMD EPYC 9V74 host and nine logical CPUs visible. Its processes are single-threaded, and the host was not isolated or CPU-pinned. Detailed environment and source

digests are stored with each result; new evidence retains its own collection provenance.

6. Results

6.1. Training and malformed-artifact conformance—All 25 distinct three-engine training profiles produced identical core bytes. The record also contains five auxiliary controls, for 30 profile executions, and one separately counted arithmetic operation-order witness. The 18 test methods exercise all four objectives, both binning methods, weighting, stratification, refitting, learning-rate schedules, repeated execution, and boundary inference. This is a finite matrix, not a branch-coverage percentage or a statistical estimate of cross-platform reliability.

The enabled early-stopping and plateau configurations in the frozen matrix do not actually trigger early termination or a plateau reduction. They verify option propagation and continued-training parity; they do not cover those positive control-flow branches. Making that distinction avoids counting a configuration as evidence for behavior it did not execute.

The malformed-artifact tests exercise unknown versions, nonzero reserved fields, inconsistent dimensions, nonpositive scale, out-of-range features, truncation, trailing bytes, metadata framing, and non-finite inputs. Eight additional executable property tests exercise traversal intervals, root-to-leaf paths, one-/two-/four-byte reads across nine chunk sizes, a three-chunk counterexample when $c < 4$, pointer capacity, rational quantization bounds, accumulator bounds, and checked-in artifact equality. All eight passed. The resulting evidence supports the named failure modes rather than exhaustive validation of every malformed byte string.

6.2. Local EVM boundary tests—The integration test uses an isolated chain. It deploys the production store, registry, model NFT, and inference runtime, then publishes a 104-byte scalar fixture and a 156-byte three-output fixture using 17-byte chunks, deliberately forcing primitive reads across boundaries. Readback reconstructs both cores exactly and verifies their content digests. Six packed vectors exercise threshold equality, neighboring branches, signed int32 extrema, and equal-logit class selection. The test also checks rejection of invalid input length.

The separate bigint evaluator matches 18 scalar/vector/class view results and two local transaction results, with zero mismatches. Equal logits select class zero as specified. The registration counterexample in Section 4 is exercised in the same run. These transactions are local test-chain operations; no experiment changes the public deployment.

A scalar scaling experiment publishes six deterministic two-feature shapes through 20,000-byte chunks and evaluates 12 seeded vectors per shape. All 72 comparisons agree with the bigint oracle. Core sizes range from 1,784 to 38,024 bytes. Recorded mean view-gas estimates range from 286,247 for 20 depth-3 trees to 3,121,210 for 200 depth-4 trees. Fifty depth-6 trees occupy more bytes than 200 depth-4 trees but require fewer decisions and a lower mean estimate, illustrating why byte size alone is insufficient to describe inference cost. The supplement retains the full scaling table; the controlled comparison below evaluates the storage decision directly.

6.3. Controlled storage comparison—The comparison stores the same six scalar cores both through the production code-as-data path and through a benchmark-only Solidity `mapping(bytes32 => bytes)`. The latter packs the canonical bytes into storage words and reads each primitive through an indexed `SLOAD`. Both paths use the same registered model, two matching registry lookups, free-view access conditions, feature vector, traversal rule, and exact accumulation. They share the compiler and Ganache/Shanghai configuration. The code

path uses 20,000-byte chunks.

The experiment measures two costs separately. Per-model publication is the sum of mined local receipts for chunk and table writes, or the corresponding packed-storage write. Common registry registration is reported separately in the result record. Inference is measured with one `eth_estimateGas` request per path and input, resetting transaction access warmth for each request. Shared contract deployment is also recorded separately: the benchmark-only comparator has a smaller feature surface, so its deployment cost cannot support a like-for-like system-cost claim.

All 72 model/input combinations produce the same result in the bigint oracle, production runtime, and packed-storage evaluator: 144 reference/EVM comparisons with zero mismatches. Readback also verifies identical stored core bytes. Every production inference estimate reproduces the corresponding observation in the earlier scalar scaling record.

Table 1. Controlled local-EVM storage comparison. Publication values are sums of mined write receipts; inference values are arithmetic means of 12 gas estimates. Code: production code-as-data implementation. Storage: packed Solidity-storage comparator. Common registry and shared deployment costs are excluded. Rounded values are shown; raw observations are archived.

T, d	Core bytes	Publication (10^6 gas)		Inference (10^3 gas)	
		Code	Storage	Code	Storage
20, 3	1,784	0.500	1.310	286.2	267.1
50, 3	4,424	1.062	3.184	644.5	597.7
50, 4	9,224	2.078	6.564	810.8	783.9
100, 4	18,424	4.003	13.012	1576.9	1518.8
200, 4	36,824	7.951	25.962	3121.2	2992.5
50, 6	38,024	8.204	26.817	1146.4	1149.5

Across these shapes, code-as-data publication consumes 61.8–69.4% less gas than the packed-storage write, equivalently a storage-to-code gas ratio of 2.62–3.27. The materialized byte footprints are nearly equal: code accounting includes chunk prefixes and the pointer table, while storage accounting includes the length slot and rounded-up data slots. Both exclude common registry metadata, executable code, account records, and trie overhead. The observed write saving is therefore a gas-schedule and implementation result, rather than a compression result.

Inference shows a different tradeoff. Packed storage uses approximately 3.3–7.3% less gas for five shapes and 0.27% more for the depth-6 shape. The production reader repeatedly resolves table entries and checks framing; the comparator reads packed words directly. Thus this is a comparison of implemented storage backends, not an isolated comparison of opcode costs. The results support lower model-publication gas for the measured code-as-data construction, but no universal inference-gas advantage. They cover one synthetic scalar family, one client, one compiler profile, and one chunk size; vector models and alternative optimized layouts remain unmeasured.

6.4. Replayable deployment observation—A fresh read-only observation selected GenesisL1 chain ID 29, block 13,602,838, timestamp 6 September 2026 at 17:20:41 UTC, and block hash `0x fd3da1020c37ee3c1fe7cd0a6060dbc5ec3ec5fb90c0b812256dc33e467dace3`.

The witness and archive collector queried <https://rpc.genesisl1.org>. Every state-dependent getter,

code read, and inference call used the selected numeric block tag. Each collection stage re-read the block hash after its calls and required it to remain unchanged. The collector additionally required the explicitly selected number/hash pair before accepting state. No public transaction was submitted.

The observer enumerated all 12 active token records, recovered their address tables and 1,306 data chunks, and reconstructed 31,185,324 core bytes. Every core passed strict format checks, all nine registry/core/storage relations, and the registered Keccak-256 commitment. The largest object contains 9,523,096 bytes in 397 chunks. Table 2 records the per-model geometry. All models use a nominal 24,000-byte chunk stride and were enabled for unrestricted view inference at the selected state.

Table 2. Public model geometry at block 13,602,838. Trees denotes total serialized trees (T or KR). Token identifiers locate the observed records; registry titles and application claims are not used as scientific labels. Every row passed content, canonical-format, manifest, and nine-relation verification.

Token	Version	F	d	Trees	Core bytes	Chunks
1	2	4	3	1,800	158,436	7
2	1	7	5	169	63,568	3
3	1	13	6	196	148,984	7
4	1	4	3	275	24,224	2
5	1	53	5	20	7,544	1
6	1	26	8	1,496	4,583,768	191
7	1	27	8	2,500	7,660,024	320
8	1	33	8	1,008	3,088,536	129
9	1	33	9	1,552	9,523,096	397
10	1	33	7	1,789	2,733,616	114
11	1	33	8	1,042	3,192,712	134
12	1	47	3	9	816	1

For each model, nine deduplicated packed vectors comprise the all-zero vector, both signed-int32 extrema, and complete $\theta - 1$, θ , $\theta + 1$ neighborhoods for two distinct non-sentinel root thresholds. Other feature coordinates remain fixed for each threshold probe. Roots are necessarily visited, so the neighborhoods exercise their named branch decisions; they do not cover every internal path. All 108 local/provider-returned scalar or vector outputs matched exactly.

The compressed evidence archive contains the model cores, exact storage runtime objects and address tables, five deployed contract runtimes, the block records, raw RPC request/response transcript, packed vectors, expected outputs, collection source, and file digests. A separate Python verifier uses only the standard library and does not import the trainers, production decoder, or JavaScript witness. It checks the file digests, parses the raw observations, reconstructs each object, recomputes Keccak-256, validates the registry relations, and evaluates every archived input with integer arithmetic. Offline replay verified all 12 models, all 31,185,324 core bytes, and all 108 output comparisons without an RPC connection.

The archive is distributed as `benchmarks/results/live_chain_archive_13602838.tar.gz`, with a small machine-readable result and checksum alongside it. Extraction and verifier commands are provided in `REPRODUCIBILITY.md`. Its raw observations support

inspection and replay of what the provider returned. Neither the transcript nor the offline verifier authenticates consensus finality, validates account-proof paths, or establishes source-to-live-bytecode identity. Contract runtime bytes and their digests are retained as observed objects, not as a verified compilation claim.

This is a new September observation. Earlier July summaries at block 13,342,043 remain available as historical records, but their missing raw responses are not reconstructed retrospectively or used as the principal deployment evidence. The present result is also bounded to one provider and one state. It does not imply that later registrations satisfy the predicate or that the 12 named applications have been predictively validated.

6.5. Reference trainer timing—The existing end-to-end timing study uses 3,000 rows, 12 deterministic features, and 60 depth-4 trees for regression and binary classification. Each engine receives one warm-up and 30 timed repetitions in rotating order. Timing includes startup, parsing, training, serialization, and output writing. Python and C++ read CSV, while Node reads an equivalent JSON matrix; the measurements therefore describe complete workflows with different parsers.

Regression medians are 0.3046 s for Python, 0.0365 s for C++, and 0.1384 s for Node; binary medians are 0.3149 s, 0.0437 s, and 0.1610 s respectively. Both workloads produce identical 11,064-byte cores across all three engines. The supplement reports interquartile ranges, and the JSON record retains all measurements. These observations establish within-host reference performance; they do not establish a general language ranking or compare GLIF with optimized external training libraries.

7. Discussion and limitations

GLIF makes a limited connection between public storage and reproducible computation explicit. Canonical decoding establishes the represented forest; ordered reconstruction establishes which bytes are stored; registry checks establish which metadata the runtime uses; and shared integer semantics establish the returned function. A digest binds the requested identifier to those bytes only when it is actually recomputed. The executable registry counterexample is useful evidence of this separation: correct execution of some referenced bytes can coexist with an incorrect content identifier.

Complete fixed-depth trees trade storage for regular control flow and direct offsets. Forced continuations can duplicate terminal values, and the current internal-node layout contains two reserved bytes per node. These choices simplify interpretation but do not minimize serialization size. Alternative sparse encodings, compressed nodes, or batched reads could change the cost balance. The storage experiment evaluates a specific packed-storage baseline under a fixed EVM configuration; it is not a proof of optimality against all possible implementations.

The formal result is conditional and handwritten. It is supported by source inspection and separate executable oracles, but the study does not prove compiler correctness or mechanically verify deployed bytecode. Positive tests cannot establish universal parser safety or training parity. In particular, differing transcendental libraries, operation reassociation, or unexercised stopping branches can change a trained core. The finite fixture matrix and retained counterexamples make those boundaries concrete.

The fresh deployment archive improves reproducibility by preserving the inputs to verification.

Offline replay can detect a corrupt archive, an inconsistent manifest, a content mismatch, or disagreement between recorded and locally computed outputs. It cannot establish that the provider supplied an authentic header or account state. Such a claim would additionally need an authenticated chain view and verified state proofs. A block-number/hash recheck detects a changed response during collection but does not remove the provider trust assumption.

The case study is developer-conducted and concerns one deployment. Separately implemented verification code reduces shared implementation dependence but is not independent authorship or external replication. Creator addresses and model titles do not establish independent adoption, dataset provenance, or predictive usefulness. Application-level evaluation would require frozen data, leakage-resistant splits, suitable baselines, uncertainty estimates, and external validation; the synthetic conformance experiments do not supply those results.

The public-byte design also limits what ownership and payment mechanisms can promise. Registry settings, fee destinations, authorization keys, and token ownership can evolve separately from the core. Their correctness must be checked for the service being used, rather than inferred from the execution theorem. A scientific result should identify both the fixed artifact and the state at which its service behavior was observed. None of the reported experiments requires a successor contract or migration of existing models.

8. Conclusion

GL1F provides a reproducible method for relating a registered public tree ensemble to the integer function it executes. Its central requirement is a composition of canonical parsing, exact ordered reconstruction, consistent runtime metadata, identical packed inputs, and exact arithmetic. The specification proof explains why these conditions suffice; the registry counterexample explains why an identifier lookup alone does not. Cross-language conformance, local EVM boundary tests, controlled storage measurements, and a replayable deployment archive provide evidence at distinct levels. The result is a bounded execution-assurance study whose checks can be repeated and whose remaining assumptions are explicit.

Data and software availability

The MIT-licensed reference implementation, canonical format specification, executable fixtures, benchmark methods, recorded results, and manuscript sources are available in the GenesisL1 Forest repository.³⁰ Third-party material retains its stated upstream terms. The supplement contains detailed proofs, field layouts, and supplementary measurements. `REPRODUCIBILITY.md` gives the pinned environment and commands. The aggregate `make verify` command runs local software checks; `make pdfs` builds the manuscript and supplement. The storage comparison and deployment archive each include the provenance required to replay their reported checks.

Competing interests

Mikhail Fedorov is the founder of GenesisL1 and the developer of GL1F. The paper evaluates software and a blockchain deployment associated with that project. This relationship is a potential competing interest.

Use of artificial intelligence

OpenAI ChatGPT/Codex assisted with manuscript drafting and revision, code inspection and corrections, test and benchmark development, and preparation of reproducibility materials. Generated content was checked through source comparison, executable tests, and numerical and document review. These checks are described with their scope and limitations in the paper and accompanying materials. The author is responsible for the submitted manuscript and its scientific claims.

Notes and References

¹ Peng, R. D. “Reproducible Research in Computational Science.” *Science* **334.6060** 1226–1227 (2011) doi:10.1126/science.1213847.

² Pineau, J., *et al.* “Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program.” *Journal of Machine Learning Research* **22.164** 1–20 (2021) URL <https://jmlr.org/papers/v22/20-303.html>.

³ Friedman, J. H. “Greedy Function Approximation: A Gradient Boosting Machine.” *The Annals of Statistics* **29.5** 1189–1232 (2001) doi:10.1214/aos/1013203451.

⁴ Chen, T., Guestrin, C. “XGBoost: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining Association for Computing Machinery* 785–794 (2016) doi:10.1145/2939672.2939785.

⁵ Ke, G., *et al.* “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems 30* Curran Associates, Inc. 3146–3154 (2017) URL <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>.

⁶ Alhaidari, A., Palanisamy, B., Krishnamurthy, P. “On-Chain Decentralized Learning and Cost-Effective Inference for DeFi Attack Mitigation.” In *7th Conference on Advances in Financial Technologies. 354 of Leibniz International Proceedings in Informatics* Schloss Dagstuhl – Leibniz-Zentrum für Informatik 35:1–35:27 (2025) doi:10.4230/LIPIcs.AFT.2025.35.

⁷ 0xSequence accessed 27 August 2026 “SSTORE2.” (n.d.) Open-source Solidity library URL <https://github.com/0xsequence/sstore2>.

⁸ ONNX Working Group accessed 29 July 2026 “ai.onnx.ml TreeEnsemble Operator, Version 5.” (2026) ONNX operator documentation URL https://onnx.ai/onnx/operators/onnx_aionnxml_TreeEnsemble.html.

⁹ Nakandala, S., *et al.* “A Tensor Compiler for Unified Machine Learning Prediction Serving.” In *14th USENIX Symposium on Operating Systems Design and Implementation* USENIX Association 899–917 (2020) URL <https://www.usenix.org/conference/osdi20/presentation/nakandala>.

¹⁰ Harris, J. D., Waggoner, B. “Decentralized and Collaborative AI on Blockchain.” In *2019 IEEE International Conference on Blockchain (Blockchain)* IEEE 368–375 (2019) doi:10.1109/BLOCKCHAIN.2019.00057.

¹¹ Drungilas, V., Vaičiukynas, E., Jurgelaitis, M., Butkienė, R., Čeponienė, L. “Towards Blockchain-Based Federated Machine Learning: Smart Contract for Model Inference.” *Applied Sciences* **11.3** 1010 (2021) doi:10.3390/app11031010.

¹² Badruddoja, S., Dantu, R., He, Y., Thompson, M. A., Salau, A., Upadhyay, K. “Making Smart Contracts Predict and Scale.” In *2022 Fourth International Conference on Blockchain Computing and Applications (BCCA)* IEEE 127–134 (2022) doi:10.1109/BCCA55292.2022.9922480.

¹³ Kadadha, M., Otrók, H., Mizouni, R., Singh, S., Ouali, A. “On-chain Behavior Prediction Machine Learning Model for Blockchain-Based Crowdsourcing.” *Future Generation Computer Systems* **136** 170–181 (2022) doi:10.1016/j.future.2022.05.025.

- ¹⁴ Li, Z., Vott, S., Krishnamachari, B. “ML2SC: Deploying Machine Learning Models as Smart Contracts on the Blockchain.” In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)* IEEE 645–649 (2024) doi:10.1109/ICBC59979.2024.10634431.
- ¹⁵ Wood, G. *Ethereum: A Secure Decentralised Generalised Transaction Ledger. Yellow paper, shanghai version, revision efc5f9a* Ethereum Project (2025) revision dated 4 February 2025. This cited revision specifies the Shanghai execution-layer version; later protocol forks are outside its scope URL <https://ethereum.github.io/yellowpaper/paper.pdf>.
- ¹⁶ Buterin, V. “EIP-170: Contract Code Size Limit.” (2016) Ethereum Improvement Proposals URL <https://eips.ethereum.org/EIPS/eip-170>.
- ¹⁷ Fu, M., Zhang, C., Hu, C., Wu, T., Dong, J., Zhu, L. “Achieving Verifiable Decision Tree Prediction on Hybrid Blockchains.” *Entropy* **25.7** 1058 (2023) doi:10.3390/e25071058.
- ¹⁸ Zhang, J., Fang, Z., Zhang, Y., Song, D. “Zero Knowledge Proofs for Decision Tree Predictions and Accuracy.” In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* ACM 2039–2053 (2020) doi:10.1145/3372297.3417278.
- ¹⁹ Singh, N., Dayama, P., Pandit, V. “Zero Knowledge Proofs Towards Verifiable Decentralized AI Pipelines.” In *Financial Cryptography and Data Security. 13411 of Lecture Notes in Computer Science* Springer 248–275 (2022) doi:10.1007/978-3-031-18283-9_12.
- ²⁰ Conway, K. D., So, C., Yu, X., Wong, K. “opML: Optimistic Machine Learning on Blockchain.” (2024) arXiv preprint doi:10.48550/arXiv.2401.17555 2401.17555 URL <https://arxiv.org/abs/2401.17555>.
- ²¹ Hildenbrandt, E., *et al.* “KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine.” In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)* IEEE 204–217 (2018) doi:10.1109/CSF.2018.00022.
- ²² Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., Cappos, J. “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes.” In *28th USENIX Security Symposium (USENIX Security 19)* USENIX Association 1393–1410 (2019) URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
- ²³ Schlegel, M., Sattler, K.-U. “Capturing End-to-End Provenance for Machine Learning Pipelines.” *Information Systems* **132** 102495 (2025) doi:10.1016/j.is.2024.102495.
- ²⁴ Lamb, C., Zacchiroli, S. “Reproducible Builds: Increasing the Integrity of Software Supply Chains.” *IEEE Software* **39.2** 62–70 (2022) doi:10.1109/MS.2021.3073045.
- ²⁵ Decentralized Science Labs *GenesisL1 Forest (GLIF): A Scientific Explanation and Technical Documentation for an On-Chain EVM Gradient-Boosted Tree Studio. Technical report* Decentralized Science Labs (2026) public protocol document dated 19 January 2026 URL <https://github.com/GenesisL1/Forest/blob/722c0513d8da2a241de2cabaad1bd8c27791b7e7/GLIF.pdf>.
- ²⁶ Entriken, W., Shirley, D., Evans, J., Sachs, N. “EIP-721: Non-Fungible Token Standard.” (2018) Ethereum Improvement Proposals URL <https://eips.ethereum.org/EIPS/eip-721>.
- ²⁷ Solidity Team version-specific documentation; accessed 27 August 2026 “Solidity 0.8.20 Documentation.” (2023) Ethereum Foundation documentation URL <https://docs.soliditylang.org/en/v0.8.20/>.
- ²⁸ ConsenSys Software Inc. archived upstream repository and package documentation; accessed 27 August 2026 “Ganache, Version 7.9.2.” (n.d.) Local Ethereum development blockchain software URL <https://github.com/trufflesuite/ganache>.
- ²⁹ Harris, C. R., *et al.* “Array Programming with NumPy.” *Nature* **585.7825** 357–362 (2020) doi:10.1038/s41586-020-2649-2.
- ³⁰ Fedorov, M. version 0.2.3, 5 September 2026 “GenesisL1 Forest (GLIF): Reference Implementation.” (2026) Software, version 0.2.3 URL <https://github.com/GenesisL1/Forest>.